

GORM + PostgreSQL

Practical Guide with Fiber CRUD

[Queries](#) • [Summary](#) • [Real-world API](#) • [NeonDB](#)

ORM	GORM v2
DB	PostgreSQL via NeonDB (serverless)
Server	Go Fiber v2
Style	Simple, no error handling, real-life routes

1. Setup

Install GORM, the PostgreSQL driver, and Fiber.

```
go mod init job-portal
go get gorm.io/gorm
go get gorm.io/driver/postgres
go get github.com/gofiber/fiber/v2
go get github.com/joho/godotenv # for .env loading
```

Model

Define your struct — GORM maps it to a table automatically.

```
type Job struct {
    ID          uint      `gorm:"primaryKey"`
    Title       string    `gorm:"not null"`
    Company     string
    Location    string
    Salary      float64
    Experience  int
    IsRemote    bool      `gorm:"default:false"`
    Applicants  int       `gorm:"default:0"`
    Status      string    `gorm:"default:'active'"`
    Department  string
    CreatedAt   time.Time
    UpdatedAt   time.Time
}
```

NeonDB Connection

Copy the connection string from your Neon dashboard. Pass it to `gorm.Open()`.

```
// .env
DATABASE_URL=postgres://user:pass@ep-xxx.neon.tech/dbname?sslmode=require

// main.go
dsn := os.Getenv("DATABASE_URL")
db, _ := gorm.Open(postgres.Open(dsn), &gorm.Config{})
db.AutoMigrate(&Job{}) // creates table if not exists
```

TIP NeonDB needs ?sslmode=require at the end of the DSN — without it the connection will be refused.

AutoMigrate

Automatically creates or updates the table schema based on your struct. Run it once on startup.

```
db.AutoMigrate(&Job{}, &Company{}, &User{}) // pass all models
```

2. Queries

2.1 Basic Find

```
var jobs []Job

db.Find(&jobs)           // all rows
db.First(&job, 1)        // by primary key
db.First(&job, "title = ?", "Engineer") // first match
db.Last(&job)            // last inserted
db.Take(&job)            // one random row
```

2.2 Where — Filtering

```
// equal
db.Where("location = ?", "Dhaka").Find(&jobs)

// not equal
db.Where("location != ?", "Dhaka").Find(&jobs)

// greater / less
db.Where("salary > ?", 70000).Find(&jobs)
db.Where("salary BETWEEN ? AND ?", 50000, 90000).Find(&jobs)

// multiple conditions (AND)
db.Where("salary > ? AND experience >= ?", 60000, 3).Find(&jobs)

// OR
db.Where("location = ? OR is_remote = ?", "Dhaka", true).Find(&jobs)

// NOT
db.Not("status = ?", "closed").Find(&jobs)

// IN list
db.Where("department IN ?", []string{"Engineering", "AI"}).Find(&jobs)

// LIKE search (case-insensitive in PostgreSQL)
db.Where("title ILIKE ?", "%engineer%").Find(&jobs)

// NULL check
db.Where("salary IS NOT NULL").Find(&jobs)
```

TIP Chain .Where() calls — each one adds an AND condition. Use a raw string for OR inside a single .Where().

2.3 Select — Pick Columns

```
db.Select("title", "company", "salary").Find(&jobs)

// scan into custom struct (lighter than full Job model)
type JobCard struct { Title string; Company string; Salary float64 }
var cards []JobCard
db.Model(&Job{}).Select("title", "company", "salary").Scan(&cards)
```

2.4 Order, Limit, Offset

```
db.Order("salary DESC").Find(&jobs)           // highest first
db.Order("salary ASC").Find(&jobs)            // lowest first
db.Order("location ASC, salary DESC").Find(&jobs)

// pagination
page, pageSize := 2, 10
db.Offset((page-1)*pageSize).Limit(pageSize).Find(&jobs)
```

2.5 Count, Pluck, Distinct

```
var total int64
db.Model(&Job{}).Where("status = ?", "active").Count(&total)

// pluck – single column into slice
var titles []string
db.Model(&Job{}).Pluck("title", &titles)

// distinct locations
var locs []string
db.Model(&Job{}).Distinct().Pluck("location", &locs)
```

2.6 Aggregation — Group, Having, Stats

```
type DeptStat struct {
    Department string
    TotalJobs  int64
    AvgSalary  float64
    MaxSalary  float64
}
var stats []DeptStat

db.Model(&Job{}).
    Select("department," +
        "COUNT(*) as total_jobs," +
        "AVG(salary) as avg_salary," +
        "MAX(salary) as max_salary").
    Where("status = ?", "active").
    Group("department").
    Having("COUNT(*) > ?", 1).
    Order("avg_salary DESC").
    Scan(&stats)
```

TIP WHERE filters raw rows before grouping. HAVING filters after grouping. Both can be used together.

2.7 Joins

```
// Preload – easiest way, no N+1 problem
db.Preload("Company").Find(&jobs)

// Manual JOIN with custom result
type Result struct { Title string; Industry string }
var res []Result
db.Model(&Job{}).
    Select("jobs.title, companies.industry").
    Joins("JOIN companies ON companies.id = jobs.company_id").
    Scan(&res)
```

2.8 Update Operations

```
// update one field
db.Model(&Job{}).Where("id = ?", 1).Update("salary", 90000)

// update multiple fields (use map to avoid zero-value skip)
db.Model(&Job{}).Where("id = ?", 1).Updates(map[string]interface{}{
    "salary": 90000, "status": "closed",
})

// increment
db.Model(&Job{}).Where("id = ?", id).
    Update("applicants", gorm.Expr("applicants + ?", 1))

// bulk update
db.Model(&Job{}).Where("department = ?", "Engineering").
    Update("salary", gorm.Expr("salary * ?", 1.10))
```

2.9 Delete (Soft & Hard)

```
// soft delete – sets deleted_at, keeps the row
db.Delete(&Job{}, id)

// hard delete – permanently removes
db.Unscoped().Delete(&Job{}, id)

// delete by condition
db.Where("status = ?", "closed").Delete(&Job{})
```

NOTE Add `gorm:"softDelete"` or embed `gorm.Model` to enable soft deletes. Without it, `Delete()` is hard by default.

3. Query Priority Summary

These are the queries you will use in almost every real project. Learn these first.

Query / Method	When to use	Priority
<code>.Find()</code>	Fetch all matching rows	MUST KNOW
<code>.First()</code> / <code>.Take()</code>	Fetch single record by ID or condition	MUST KNOW
<code>.Where("field = ?", val)</code>	Filter rows — the backbone of every query	MUST KNOW
<code>.Where("field ILIKE ?", v)</code>	Case-insensitive search — search bars, filters	MUST KNOW
<code>.Create()</code>	Insert a new record	MUST KNOW
<code>.Update()</code> / <code>.Updates(map)</code>	Modify fields — always use map for multiple fields	MUST KNOW
<code>.Delete()</code>	Remove a record (soft or hard)	MUST KNOW
<code>.Count()</code>	Total number of matching rows	MUST KNOW
<code>.Order()</code> + <code>.Limit()</code> + <code>.Offset()</code>	Sorting + pagination — every list endpoint needs this	MUST KNOW
<code>.Select()</code> + <code>.Scan()</code>	Pick only the columns you need into a custom struct	IMPORTANT
<code>.Group()</code> + <code>.Having()</code>	Aggregation — dashboards, stats, reports	IMPORTANT
<code>.Preload()</code>	Load related records without N+1 queries	IMPORTANT
<code>.Joins()</code>	SQL JOIN for filtered cross-table queries	IMPORTANT
<code>gorm.Expr("col + ?", n)</code>	Increment/decrement safely (applicants, views)	IMPORTANT
<code>.Where("IN ?")</code>	Match against a list of values	IMPORTANT
<code>.Not()</code> / <code>.Or()</code>	Exclude or alternate conditions	GOOD TO KNOW
<code>.Distinct().Pluck()</code>	Get unique values of a single column	GOOD TO KNOW
<code>.Where("BETWEEN ? AND ?")</code>	Salary/date range filtering	GOOD TO KNOW
<code>.Unscoped().Delete()</code>	Permanently delete a soft-deleted record	GOOD TO KNOW
<code>db.Transaction(func(tx){})</code>	Multi-step operations that must all succeed	GOOD TO KNOW

NOTE Use `map[string]interface{}` with `.Updates()` — struct skips zero values (0, false, empty string).

4. Fiber CRUD — Full Working API

Job Portal API using Go Fiber + GORM + NeonDB (PostgreSQL). Includes search, filters, pagination, stats, and all standard CRUD routes. No extra error handling — kept minimal and readable.

main.go

[illegible]

[illegible]

Create / Get One / Update / Delete

```
// POST /api/jobs - create
func createJob(c *fiber.Ctx) error {
    job := new(Job)
    c.BodyParser(job)
    DB.Create(job)
    return c.Status(201).JSON(job)
}
```


List with Filters + Pagination

```
// GET /api/jobs?page=1&limit=10&location=Dhaka&min_salary=50000
func getJobs(c *fiber.Ctx) error {
    var jobs []Job
    query := DB.Model(&Job{})

    // optional filters
    if loc := c.Query("location"); loc != "" {
        query = query.Where("location = ?", loc)
    }
    if dep := c.Query("department"); dep != "" {
        query = query.Where("department = ?", dep)
    }
    if min := c.QueryFloat("min_salary", 0); min > 0 {
        query = query.Where("salary >= ?", min)
    }
    if remote := c.Query("remote"); remote == "true" {
        query = query.Where("is_remote = ?", true)
    }

    // pagination
    page := c.QueryInt("page", 1)
    limit := c.QueryInt("limit", 10)
    var total int64
    query.Count(&total)
    query.Order("created_at DESC").
        Offset((page-1)*limit).Limit(limit).Find(&jobs)

    return c.JSON(fiber.Map{
        "data": jobs,
        "total": total,
        "page": page,
    })
}

// GET /api/jobs/:id
func getJob(c *fiber.Ctx) error {
    var job Job
    DB.First(&job, c.Params("id"))
    return c.JSON(job)
}

// PUT /api/jobs/:id
func updateJob(c *fiber.Ctx) error {
    var job Job
    DB.First(&job, c.Params("id"))
    c.BodyParser(&job)
    DB.Save(&job)
    return c.JSON(job)
}

// DELETE /api/jobs/:id
func deleteJob(c *fiber.Ctx) error {
    DB.Delete(&Job{}, c.Params("id"))
    return c.JSON(fiber.Map{"message": "deleted"})
}
```

Extra Real-Life Routes

```
// GET /api/jobs/search?q=react
func searchJobs(c *fiber.Ctx) error {
    var jobs []Job
    q := "%" + c.Query("q") + "%"
    DB.Where("title ILIKE ? OR company ILIKE ? OR department ILIKE ?",
        q, q, q).Order("created_at DESC").Find(&jobs)
    return c.JSON(jobs)
}

// PATCH /api/jobs/:id/apply - increment applicant count
func applyJob(c *fiber.Ctx) error {
    DB.Model(&Job{}).Where("id = ?", c.Params("id")).
        Update("applicants", gorm.Expr("applicants + ?", 1))
    return c.JSON(fiber.Map{"message": "applied"})
}

// PUT /api/jobs/:id/status body: {"status": "closed"}
func changeStatus(c *fiber.Ctx) error {
    body := struct{ Status string `json:"status"` }{}
    c.BodyParser(&body)
    DB.Model(&Job{}).Where("id = ?", c.Params("id")).
        Update("status", body.Status)
    return c.JSON(fiber.Map{"status": body.Status})
}
```

```
// GET /api/jobs/remote
func remoteJobs(c *fiber.Ctx) error {
    var jobs []Job
    DB.Where("is_remote = ? AND status = ?", true, "active").
        Order("salary DESC").Find(&jobs)
    return c.JSON(jobs)
}

// GET /api/jobs/top-applicants?limit=5
func topApplicants(c *fiber.Ctx) error {
    var jobs []Job
    limit := c.QueryInt("limit", 5)
    DB.Order("applicants DESC").Limit(limit).Find(&jobs)
    return c.JSON(jobs)
}
```

Stats Routes

```
// GET /api/jobs/stats – overall numbers
func jobStats(c *fiber.Ctx) error {
    var total, active, remote int64
    var avgSalary float64

    DB.Model(&Job{}).Count(&total)
    DB.Model(&Job{}).Where("status = ?", "active").Count(&active)
    DB.Model(&Job{}).Where("is_remote = ?", true).Count(&remote)
    DB.Model(&Job{}).Select("AVG(salary)").Scan(&avgSalary)

    return c.JSON(fiber.Map{
        "total_jobs": total,
        "active_jobs": active,
        "remote_jobs": remote,
        "avg_salary": avgSalary,
    })
}
```

```
// GET /api/jobs/stats/dept – per-department breakdown
func statsByDept(c *fiber.Ctx) error {
    type DeptStat struct {
        Department string `json:"department"`
        TotalJobs int64 `json:"total_jobs"`
        AvgSalary float64 `json:"avg_salary"`
        MaxSalary float64 `json:"max_salary"`
    }
    var stats []DeptStat

    DB.Model(&Job{}).
        Select("department," +
            "COUNT(*) as total_jobs," +
            "AVG(salary) as avg_salary," +
            "MAX(salary) as max_salary").
        Where("status = ?", "active").
        Group("department").
        Order("avg_salary DESC").
        Scan(&stats)

    return c.JSON(stats)
}
```

API Route Reference

Method	Route	Description
POST	/api/jobs	Create a new job
GET	/api/jobs	List jobs (filter + paginate)
GET	/api/jobs/:id	Get one job by ID
PUT	/api/jobs/:id	Update a job
DELETE	/api/jobs/:id	Delete a job
GET	/api/jobs/search?q=...	Full-text search across title, company, dept

Method	Route	Description
PATCH	/api/jobs/:id/apply	Increment applicants count by 1
PUT	/api/jobs/:id/status	Change job status (active/closed)
GET	/api/jobs/remote	All active remote jobs, sorted by salary
GET	/api/jobs/top-applicants	Top N most-applied jobs
GET	/api/jobs/stats	Overall counts and average salary
GET	/api/jobs/stats/dept	Per-department stats

.env file

```
DATABASE_URL=postgres://user:password@ep-something.neon.tech/dbname?sslmode=require
```

Run

```
go run main.go
# server starts on :8080

# test
curl -X POST http://localhost:8080/api/jobs \
  -H "Content-Type: application/json" \
  -d '{"title":"Backend Dev","company":"TechCorp","salary":75000,"department":"Engineering"}'

curl "http://localhost:8080/api/jobs?location=Dhaka&min_salary=60000&page=1&limit=5"
curl "http://localhost:8080/api/jobs/search?q=react"
curl "http://localhost:8080/api/jobs/stats"
```